

MODUL 3 - Blade Template Laravel



Disusun Oleh:

Akmallutfi Afza Gusty

707012500064

SIKC 49 - 04

Mata Kuliah:

Pengembangan Aplikasi Berbasis Web

Dosen Pengampu:

Diash Firdaus

1. Tujuan Praktikum

Setelah mengikuti praktikum ini mahasiswa diharapkan dapat:

1. Mengenal konsep Blade Template di Laravel.
2. Memahami sintaks dasar Blade.
3. Mampu menggunakan direktif (directive) Blade untuk percabangan dan perulangan.
4. Mampu menggunakan fitur layouting dengan `@extends`, `@section`, dan `@yield`.
5. Menggunakan komponen Blade (`@include` dan komponen custom) untuk membuat tampilan dinamis.
6. Menerapkan Blade untuk men-generate konten web yang terstruktur dan mudah dikelola.

2. Alat dan Bahan

Untuk menjalankan modul ini, diperlukan:

Perangkat Keras:

- Komputer atau laptop dengan sistem operasi Windows/Linux/macOS.
- Minimal prosesor 1 GHz, RAM 4GB (disarankan 8GB), serta ruang penyimpanan kosong minimal 2GB.

Perangkat Lunak:

- XAMPP atau Laragon sebagai server lokal (Apache & MySQL).
- Composer sebagai dependency manager PHP.
- PHP versi 8.0 atau lebih tinggi (sudah termasuk dalam XAMPP/Laragon).
- Laravel Framework (minimal versi 8) yang sudah terinstal melalui Composer.
- Teks Editor / IDE seperti Visual Studio Code, Sublime Text, atau PHPStorm.
- Browser Web seperti Google Chrome, Safari, atau Mozilla Firefox untuk menjalankan aplikasi.

3. Langkah-Langkah Praktikum

3.1. Konfigurasi Route

Langkah pertama adalah mendefinisikan rute (route) aplikasi pada file **routes/web.php**. Rute ini menghubungkan URL yang diakses di browser dengan method pada Controller. Pada praktikum ini dibuat route **GET /students** yang diarahkan ke method **index()** pada **StudentController**, dan diberi nama **students.index** agar dapat dipanggil kembali di dalam view menggunakan helper **route()**.

```
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\StudentController;

Route::get('/', function () {
```

```

        return view('welcome');
    });

Route::get('/students', [StudentController::class, 'index'])->name('students.index');

```

Penjelasan:

- **use App\Http\Controllers\StudentController;** — mengimpor controller agar class tersebut dapat digunakan pada file route.
- **Route::get('/students', [StudentController::class, 'index'])** — membuat route GET /students dan menjalankan method index() ketika URL tersebut dibuka.
- **->name('students.index')** — memberi nama pada route, sehingga di dalam Blade dapat dipanggil dengan **route('students.index')**.

3.2. Pembuatan Controller

Controller dibuat dengan perintah **php artisan make:controller StudentController**, kemudian isinya diganti dengan kode berikut. Controller berperan sebagai penyiap data: method **index()** mendefinisikan data mahasiswa dalam bentuk array multidimensi (nama, jurusan, umur, dan daftar mata kuliah) lalu mengirimkannya ke view. Karena masih berupa prototipe, data belum diambil dari database. Fungsi **compact('students')** setara dengan **['students' => \$students]**, sehingga di dalam view data dapat dipanggil melalui variabel **\$students**.

```

<?php

namespace App\Http\Controllers;
use Illuminate\Http\Request;

class StudentController extends Controller
{
    public function index(){
        $students = [[
            'name' => 'yudi',
            'major' => 'sikc',
            'age' => 23,
            'courses' => ['Pemrograman Web', 'Database',
'Cloude Computing']
        ],[
            'name' => 'Siti',
            'major' => 'Manajemen',
            'age' => 21,
            'courses' => ['Manajemen Data', 'Akutansi',
'Matematika']
        ]
    ];

    return view('students.index', compact('students'));
}

```

```
}  
}
```

3.3. Pembuatan Layout Utama

Selanjutnya dibuat file layout utama **resources/views/layouts/main.blade.php**. Layout ini merupakan kerangka halaman yang selalu sama pada setiap halaman, yaitu **<head>**, CSS, header, menu navigasi, dan footer. Bagian yang berubah ditandai dengan directive **@yield('title')** untuk judul halaman dan **@yield('content')** untuk isi konten. Menu navigasi menggunakan named route **route('students.index')**, sehingga tautan tetap benar walaupun URL diubah.

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
    <meta charset="UTF-8">  
    <meta name="viewport" content="width=device-width,  
initial-scale=1.0">  
    <title>@yield('title') | Mantap</title>  
  
    <style>  
        body{font-family: Arial, Helvetica, sans-serif;  
margin: 20px;}  
        header, footer{background: #f5f5f5; padding: 10px;  
text-align: center;}  
        nav a {margin: 0 10px; text-decoration: none;}  
        .card {border: 1px solid #ddd; padding: 15px; margin:  
10px 0; border-radius: 6px;}  
        .course {background: #eef; margin: 3px; padding: 5px;  
display: inline-block; border-radius: 4 px;}  
    </style>  
</head>  
<body>  
    <header>  
        <h1>Mantap</h1>  
        <nav>  
            <a  
href="{{route('students.index')}}">Mahasiswa</a>  
        </nav>  
    </header>  
  
    <main>  
        @yield('content')  
    </main>  
  
    <footer>  
        <p>&copy; 2027 praktikum laravel - Akmallutfi Afza  
Gusty</p>  
    </footer>
```

```
</body>
</html>
```

3.4. Pembuatan View Daftar Mahasiswa

View **resources/views/students/index.blade.php** mewarisi layout utama dengan **@extends('layouts.main')**, kemudian mengisi bagian judul melalui **@section('title', ...)** dan bagian konten melalui **@section('content')**. Data mahasiswa ditampilkan menggunakan directive **@foreach**, status ditentukan dengan percabangan **@if** (umur ≥ 22 berstatus "Senior", selain itu "Junior"), dan daftar mata kuliah ditampilkan dengan **@forelse** sehingga dapat menampilkan pesan **@empty** apabila tidak ada mata kuliah yang terdaftar.

```
@extends('layouts.main')

@section('title', 'Daftar Mahasiswa')

@section('content')
<h2>Daftar Mahasiswa</h2>

<x-alert type="success" message="Data Mahasiswa Berhasil
dimuat"></x-alert>

@foreach($students as $student)
<div class="card">
    <h3>{{ $student['name'] }}</h3>
    <p><strong>Jurusan:</strong> {{ $student['major'] }}</p>
    <p><strong>Umur:</strong> {{ $student['age'] }} tahun</p>

    @if($student['age'] >= 22)
    <p>Status: <span style="color: green;">Senior</span></p>
    @else
    <p>Status: <span style="color: blue;">Junior</span></p>
    @endif

    <h4>Mata Kuliah:</h4>

    @forelse($student['courses'] as $course)
    <span class="course">{{ $course }}</span>
    @empty
    <p>Tidak ada mata kuliah yang terdaftar.</p>
    @endforelse
</div>
@endforeach
@endsection
```

3.5. Pembuatan Komponen Blade (Alert)

Komponen Blade dibuat pada file **resources/views/components/alert.blade.php**. File di dalam folder **components** otomatis dikenali sebagai komponen dan dipanggil dengan tag berawalan **x-**, misalnya **<x-alert>**. Data dikirim ke komponen melalui atribut **type** dan **message**; operator ternary **\$type === 'error' ? 'red' : 'green'** menentukan warna border, dan **ucfirst(\$type)** mengubah huruf pertama tipe menjadi kapital.

```
<div style="padding: 10px; border:1px solid {{ $type ===  
'error' ? 'red' : 'green' }}; margin : 10 px 0;">  
  
<strong>{{ucfirst($type)}}:</strong>{{ $message}}  
</div>
```

Komponen alert dipanggil pada view **students/index.blade.php** sebagai berikut:

```
<x-alert type="success" message="Data Mahasiswa Berhasil  
dimuat"></x-alert>
```

Karena dibuat sebagai komponen yang dapat dipakai berulang (reusable), pemanggilan dengan **type="success"** menghasilkan alert berwarna hijau, sedangkan **type="error"** menghasilkan alert berwarna merah tanpa perlu menulis ulang struktur HTML-nya.

3.6. Menjalankan Aplikasi

Setelah seluruh file disimpan, server pengembangan dijalankan melalui terminal, kemudian halaman diakses di browser.

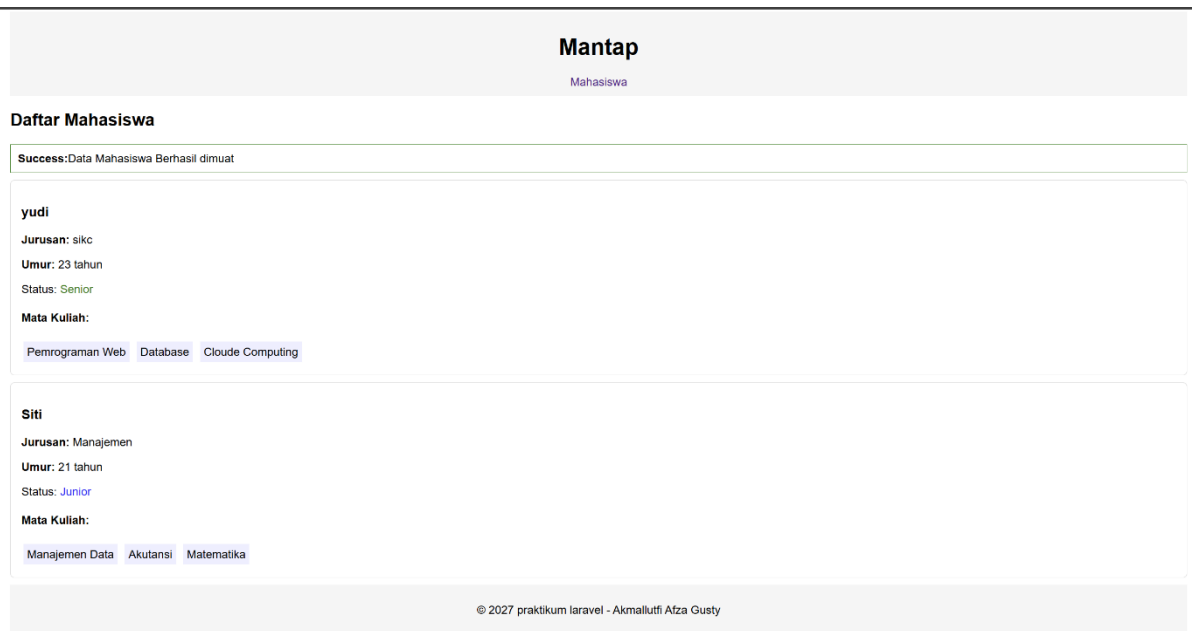
```
php artisan serve
```

Halaman daftar mahasiswa dapat dibuka melalui alamat **http://127.0.0.1:8000/students**.

4. Hasil dan Dokumentasi Program

Setelah seluruh file (route, controller, layout, view, dan komponen) dibuat dan disimpan, aplikasi dijalankan dengan perintah **php artisan serve**, lalu halaman diakses melalui alamat **http://127.0.0.1:8000/students**. Berikut merupakan hasil tampilan aplikasi yang telah dibuat.

Gambar 4.1. Tampilan Halaman Daftar Mahasiswa



Keterangan Gambar 4.1: Menampilkan halaman daftar mahasiswa yang dihasilkan dari view **students/index.blade.php**. Terlihat header "Mantap" beserta menu navigasi "Mahasiswa", komponen alert berwarna hijau di bagian atas, serta dua kartu mahasiswa (yudi dan Siti) yang menampilkan nama, jurusan, umur, status, dan daftar mata kuliah.

Penjelasan hasil tampilan:

- **Komponen alert:** dipanggil dengan `<x-alert type="success" message="Data Mahasiswa Berhasil dimuat">` sehingga muncul kotak pesan berwarna hijau dengan teks "Success: Data Mahasiswa Berhasil dimuat".
- **Perulangan dan percabangan:** setiap kartu mahasiswa dihasilkan oleh directive `@foreach`, sedangkan status pada kartu ditentukan oleh `@if` (umur ≥ 22 menampilkan "Senior" berwarna hijau, selain itu "Junior" berwarna biru).
- **Daftar mata kuliah:** ditampilkan dengan `@forelse` dalam bentuk label berwarna; apabila daftar kosong akan muncul pesan "Tidak ada mata kuliah yang terdaftar."
- **Layouting:** bagian header, menu navigasi, dan footer berasal dari layout utama **layouts/main.blade.php**, sehingga seluruh halaman memiliki tampilan yang seragam.

Berkas-berkas yang dihasilkan pada praktikum ini adalah **routes/web.php**, **app/Http/Controllers/StudentController.php**, **resources/views/layouts/main.blade.php**, **resources/views/students/index.blade.php**, dan **resources/views/components/alert.blade.php**.

5. Analisis dan Pembahasan

Pada praktikum ini, penerapan Blade Template pada pola MVC berjalan dengan baik. Data disiapkan oleh Controller, dikirim ke View, dan ditampilkan secara dinamis menggunakan directive Blade. Berikut beberapa hal yang dapat dianalisis dari hasil praktikum.

- **Pemisahan Route, Controller, dan View:** route pada **routes/web.php** hanya bertugas memetakan URL **/students** ke method **index()**. Controller

StudentController bertugas menyiapkan data **\$students**, sedangkan view sepenuhnya fokus menampilkan data. Dengan pemisahan ini tidak ada kode logika PHP di dalam view, sehingga kode lebih terstruktur, mudah dibaca, dan mudah diperbaiki.

- **Pemanfaatan Layouting (@extends, @section, @yield):** kerangka halaman yang selalu sama (head, CSS, header, navigasi, dan footer) cukup ditulis satu kali pada **layouts/main.blade.php**, sedangkan bagian yang berbeda diisi oleh view turunan melalui **@section** dan ditampilkan di layout melalui **@yield**. Cara ini menghindari penulisan ulang kode HTML; apabila ingin mengubah menu atau warna tampilan, cukup mengubah satu file layout saja.
- **Directive untuk percabangan dan perulangan: @foreach** digunakan untuk menampilkan seluruh data mahasiswa, **@if/@else** untuk menentukan status "Senior"/"Junior" berdasarkan umur, serta **@forelse** dan **@empty** untuk menampilkan daftar mata kuliah sekaligus menangani kondisi ketika daftar kosong. HTML kartu mahasiswa pun di-generate otomatis dari data, bukan ditulis manual satu per satu.
- **Reusable Component Alert:** komponen **<x-alert>** dibuat satu kali pada **components/alert.blade.php** dan dapat dipanggil berkali-kali dengan nilai atribut **type** dan **message** yang berbeda. Operator ternary menentukan warna border dan **ucfirst()** merapikan huruf awal tipe. Ketika desain alert ingin diubah, cukup mengubah satu file komponen dan seluruh pemanggilnya ikut berubah.
- **Kendala dan Penyelesaian:** beberapa galat yang sempat muncul antara lain (1) "Route [students.index] not defined" karena nama route belum ditambahkan — diselesaikan dengan menambahkan **->name('students.index')**; (2) "Undefined variable \$students" karena nama variabel pada **compact()** tidak sesuai — diselesaikan dengan menyamakan nama variabel; (3) parse error akibat directive tidak ditutup (**@if** tanpa **@endif**, **@foreach** tanpa **@endforeach**) — diselesaikan dengan memastikan setiap directive memiliki penutupnya; dan (4) "View [students.index] not found" karena penamaan folder/file salah — diselesaikan dengan memastikan file berada pada **resources/views/students/index.blade.php**.

6. Kesimpulan

Praktikum ini berhasil menunjukkan cara kerja alur Request-Response di Laravel menggunakan Route, Controller, dan View. Melalui Blade template engine, data berhasil diproses dari form oleh Controller dan dikirim secara dinamis ke View. Pemanfaatan resource statis (CSS dan JavaScript kustom) membuktikan bahwa tampilan suatu sistem informasi web dapat dibuat menjadi interaktif dan modern tanpa mengubah satupun baris logika algoritma pemrosesan yang ada pada sisi backend.

7. Lampiran

Tautan Repositori GitHub:

[MASUKKAN LINK REPOSITORI GITHUB ANDA DI SINI]